



REPORTA TU CALLE

Entrega Final – Desarrollo de Software

— INTEGRANTES —



Chontay Campos, Bryan Aldrin



Leon Gonzales, Paul Fran



Osorio Miranda, Arturo Miguel



Sánchez Longa, Erick Joel

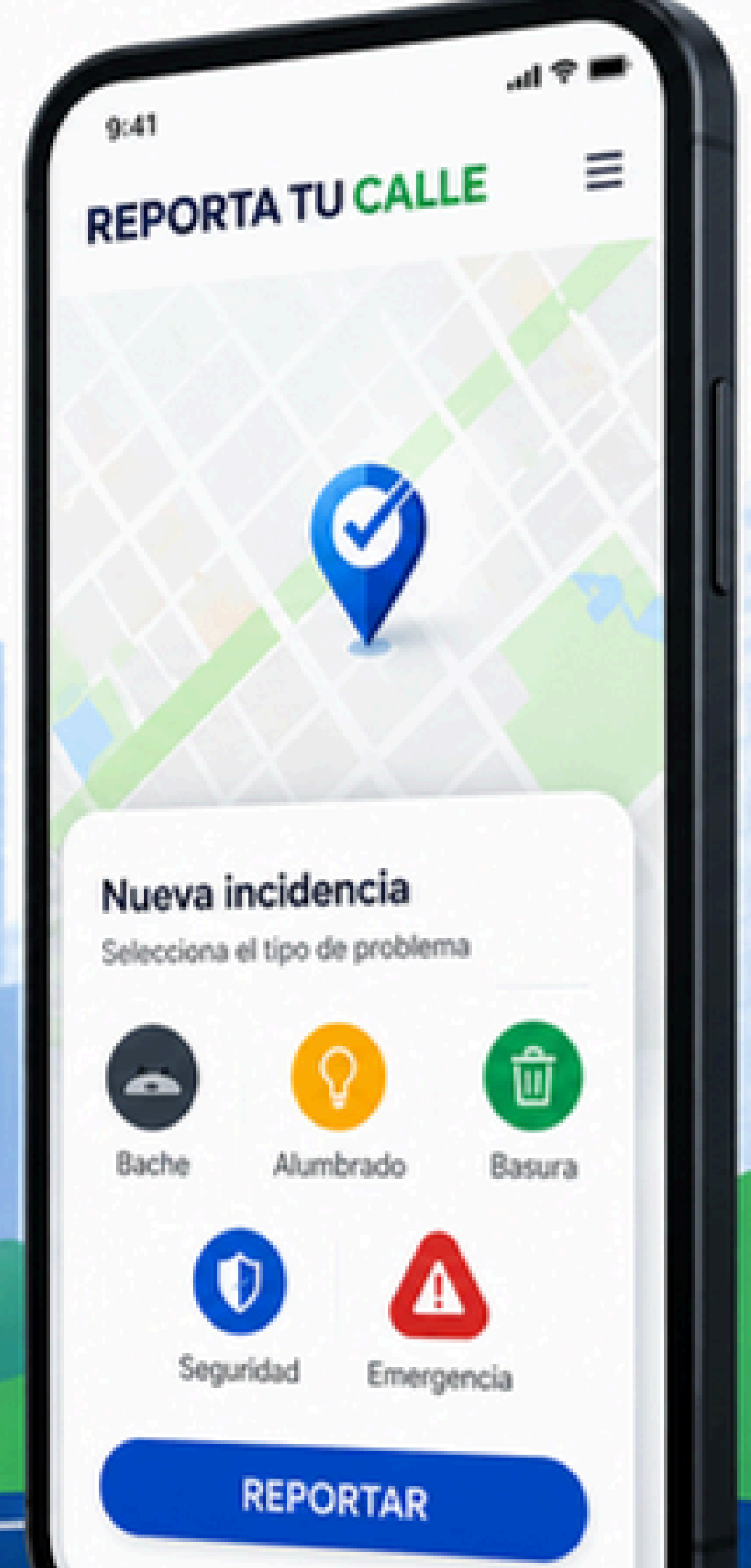


Torres Reátegui, Joaquin Marcelo



PROFESOR:

Manuel Quispe Torres



ÍNDICE

- 01** Problema y Solución Propuesta
- 02** Arquitectura de Software y tecnologías
- 03** Desarrollo Frontend
- 04** Patrones de Diseño
- 05** Estrategia de Testing
- 06** Demostración del aplicativo



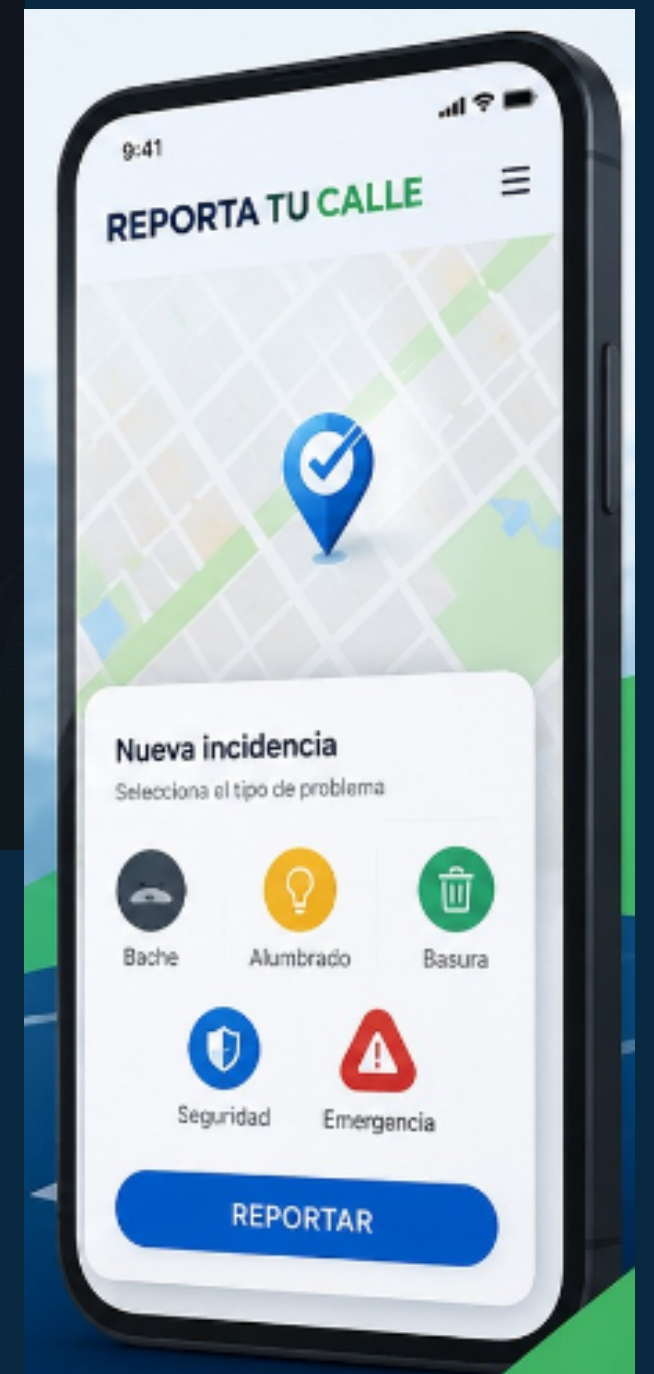
01

El problema y la solución

Problema

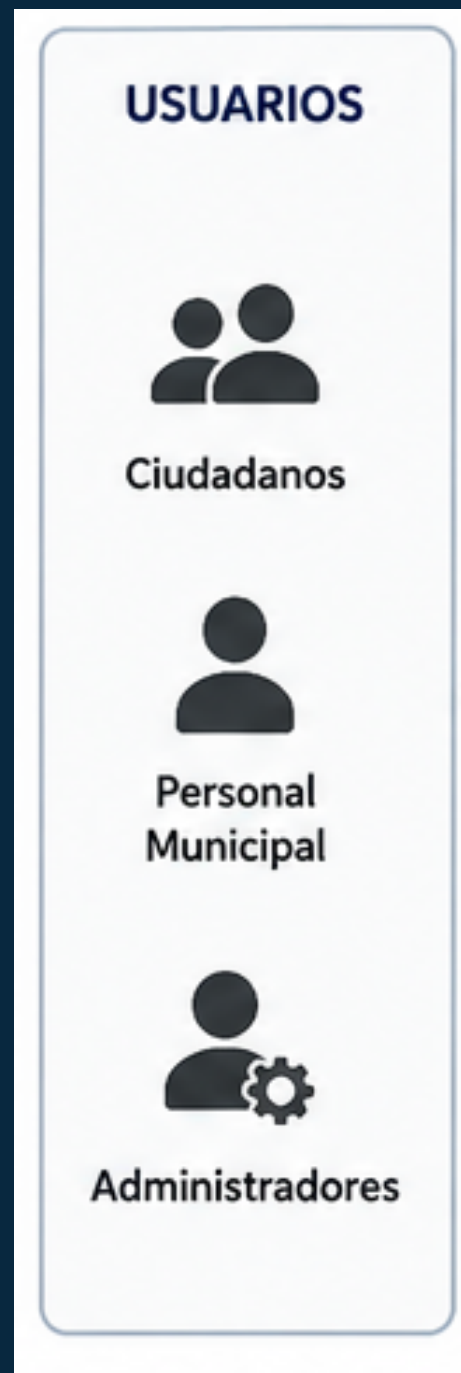


Solución



☒	RTC - 44	Levantamiento de los requerimientos principales, especificación del ciclo de vida del reporte y definición de reglas de optimi...	FINALIZADA ▾	-	
☒	RTC - 45	Evolución de Arquitectura Hexagonal y Desacoplamiento estructural mediante Patrones de Diseño GoF	FINALIZADA ▾	-	
☒	RTC - 46	Integración de Servicios UI Principales: Estandarización de Envoltorios API, WebSockets STOMP y Componentes Reactivos	FINALIZADA ▾	-	
☒	RTC - 47	Aseguramiento de Calidad (Backend): Diseño e Implementación de Suite Extensiva de Pruebas Unitarias con MockMvc y Mo...	FINALIZADA ▾	-	
☒	RTC - 48	Ingeniería de Pruebas Automáticas E2E (End-to-End): Automatización de Flujos Críticos con Cypress	FINALIZADA ▾	-	

Roles



Ciudadanos

- Registran incidencias urbanas.
- Adjuntan descripción, ubicación e imágenes.
- Consultan el estado de sus reportes.
- Visualizan incidencias en el mapa.

Supervisor (Personal Municipal)

- Registran incidencias urbanas.
- Actualiza el estado del reporte (en proceso/resuelto)
- Consulta la ruta óptima entre un conjunto de incidencias

Administrador

Gestiona usuarios, roles y categorías urbanas.
El administrador registra y administra supervisores.
Puede asignar incidencias o zonas de trabajo.

Panel administrador

Panel de Administración

Supervisa reportes, asigna tareas y gestiona usuarios.

Reportes

Usuarios

Categorías

Filtrar reportes por Supervisor:

Todos los reportes

ID	Reporte	Estado	Asignado a
#3	poda urgente -77.8442, -34.7888	Resuelto	supervisor@test.com (Supervisor Norte)
#2	Semaforo averiado por Av.Miguel Grau -12.0596, -77.0347	Pendiente	Sin asignar Sin asignar supervisor@test.com (Supervisor Norte) supervisor2@test.com (Supervisor Sur) supervisor3@test.com (Supervisor Este)
#1	Bache profundo en Jr.Moquegua 755 -12.0464, -77.0428	Pendiente	

Panel de Administración

Supervisa reportes, asigna tareas y gestiona usuarios.

Reportes

Usuarios

Categorías

AÑADIR NUEVO SUPERVISOR

Escribe el correo exacto del ciudadano a promover

Ascender a Supervisor

Equipo de Trabajo (Staff)

Buscar personal

USUARIO	ROL ACTUAL	ACCIONES
admin@test.com	Administrador	Protegido
supervisor@test.com	Supervisor	Revocar Permisos
admin2@test.com	Administrador	Protegido
admin3@test.com	Administrador	Protegido
supervisor2@test.com	Supervisor	Revocar Permisos
supervisor3@test.com	Supervisor	Revocar Permisos

Panel administrador

Panel de Administración

Supervisa reportes, asigna tareas y gestiona usuarios.

Reportes

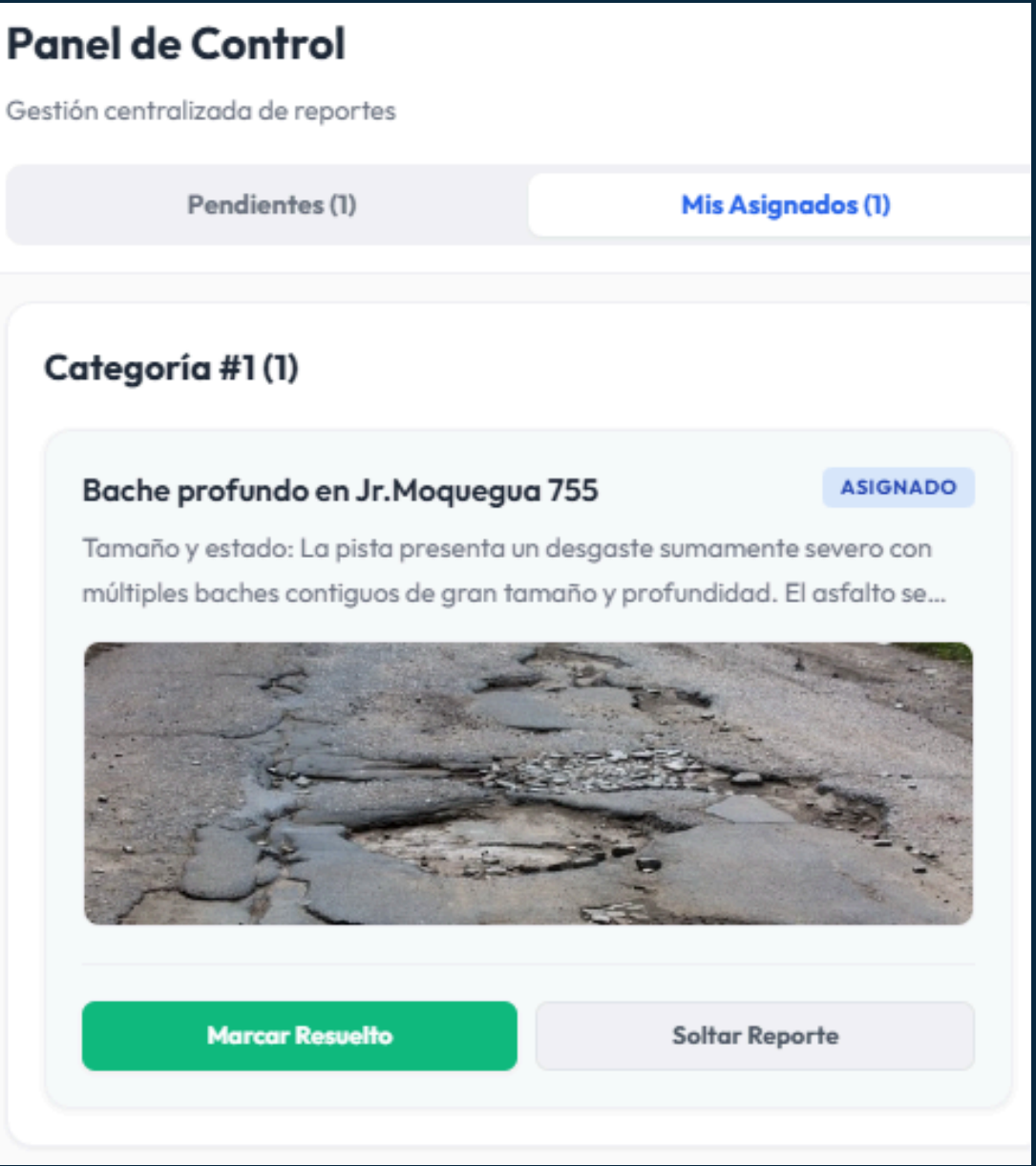
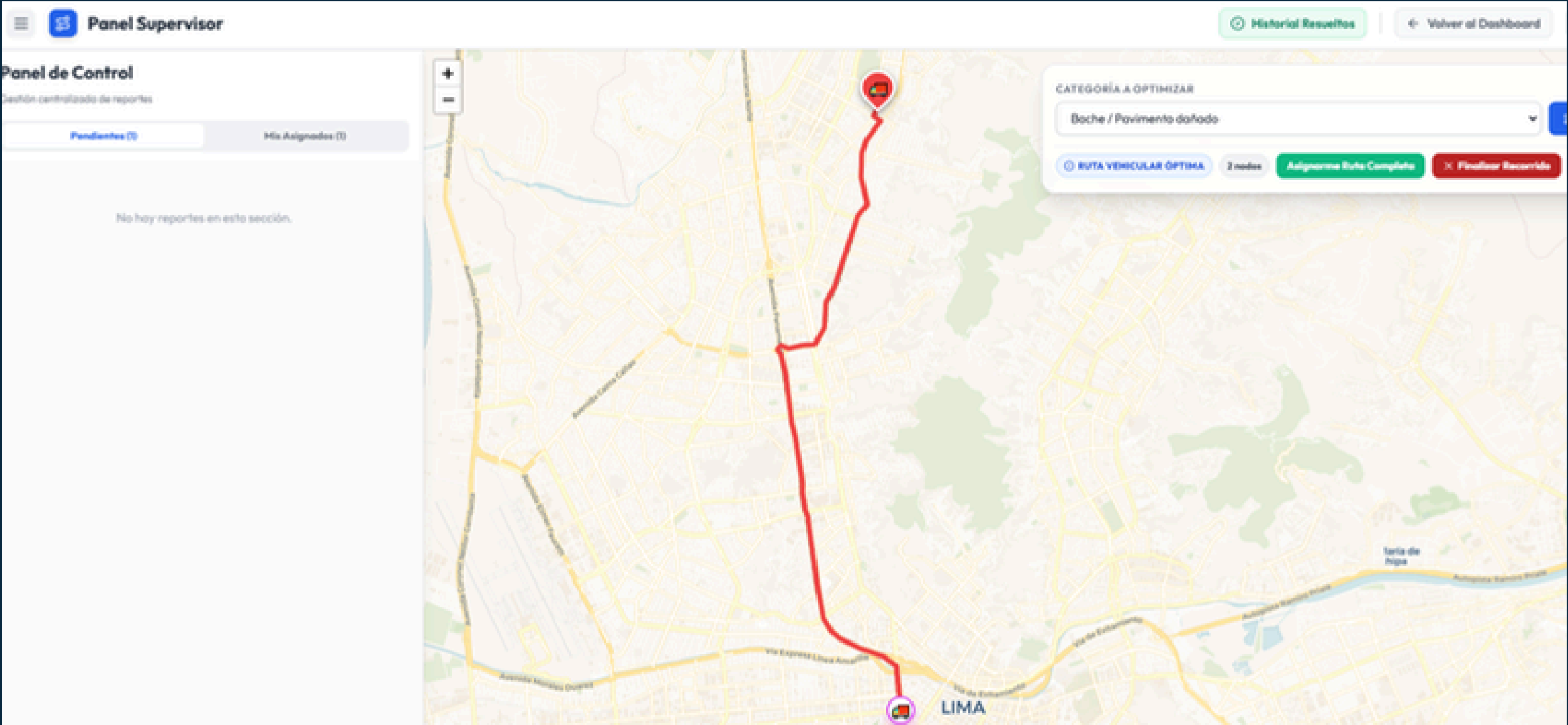
Usuarios

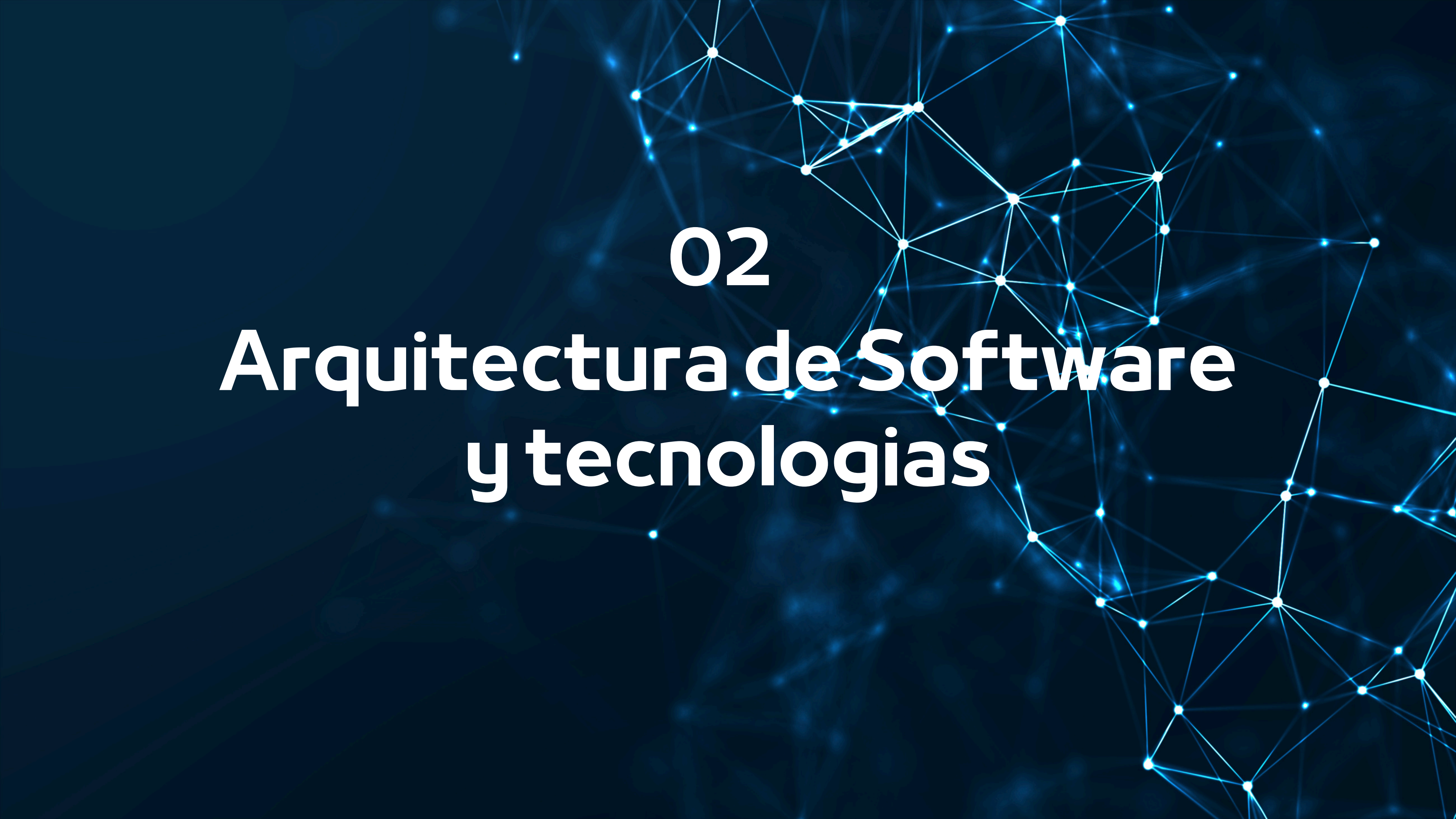
Categorías

+ Nueva categoría

Árbol caído / Poda urgente Árbol o rama que obstruye la vía o representa riesgo.	Visualización		
Bache / Pavimento dañado Huecos o grietas en el asfalto que representan peligro.	Rutas		
Basura acumulada Acumulación de residuos en vía pública o espacios comunes.	Rutas		
Fuga de agua Pérdida de agua en tuberías o conexiones visibles en la vía.	Flujo máximo		
Otro Incidencia que no encaja en las categorías anteriores.	Visualización		
Poste / Alumbrado público Luminaria apagada, poste caído o con riesgo eléctrico.	Conectividad		
Semáforo averiado Semáforo sin funcionar o con señal incorrecta.	Conectividad		
Vereda / Acera deteriorada Rotura o hundimiento de la acera que dificulta el tránsito.	Rutas		

Panel de supervisor



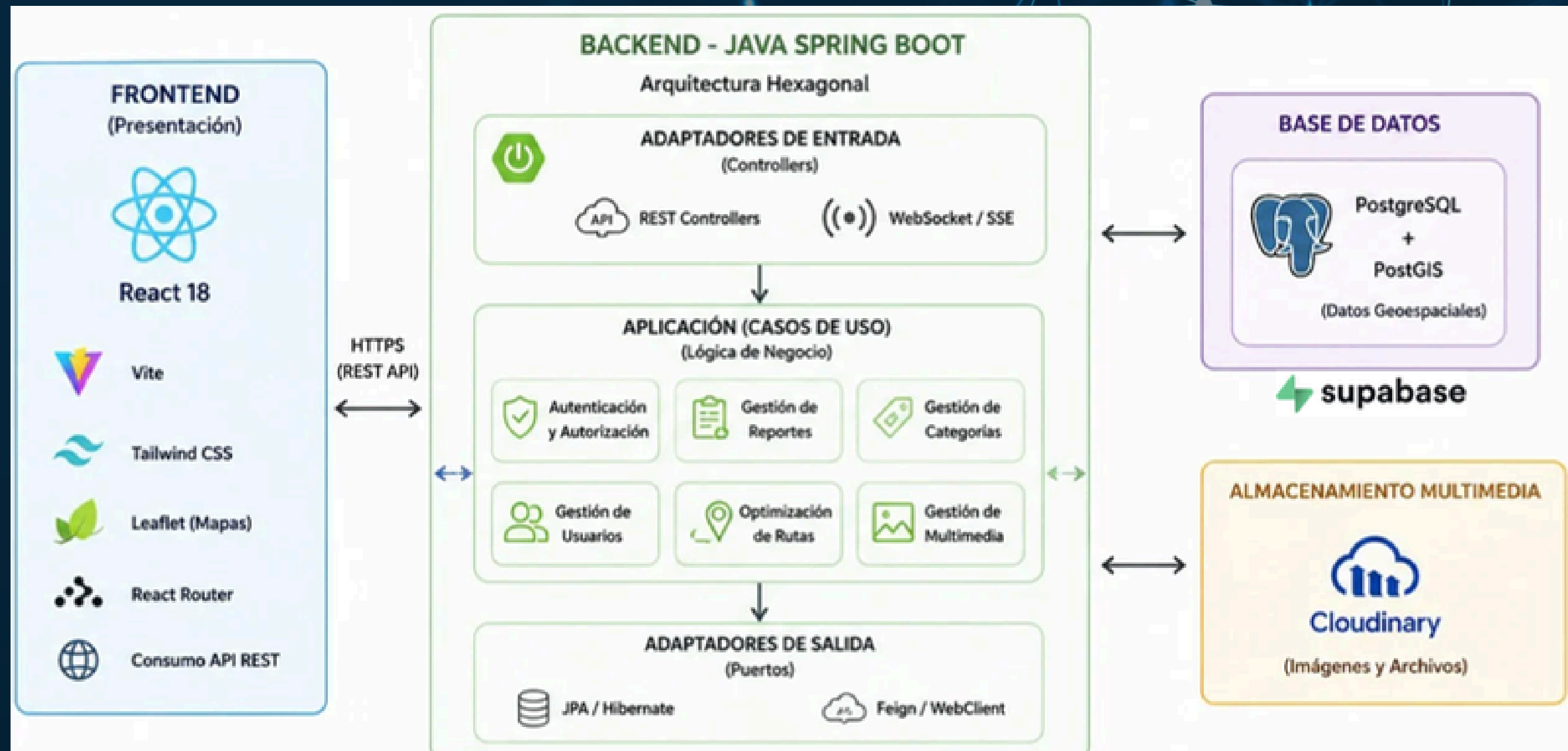


02

Arquitectura de Software y tecnologías

El sistema está construido bajo una arquitectura hexagonal, separando claramente la lógica de negocio, los casos de uso, los controladores REST y los adaptadores de persistencia o servicios externos

- **Frontend:** React 18, Vite, Tailwind CSS, Leaflet y React Router.
- **Backend:** Java 21, Spring Boot 3, Spring Security, JWT, JPA/Hibernate y WebSocket/SSE.
- **Base de datos:** PostgreSQL con PostGIS para manejo de datos geospaciales.
- **Multimedia:** Cloudinary para almacenamiento de imágenes.
- **Calidad:** JUnit, Mockito, Cypress, Locust y SonarQube.
- **Despliegue:** Frontend preparado para Vercel y Backend dockerizado/preparado para Render.

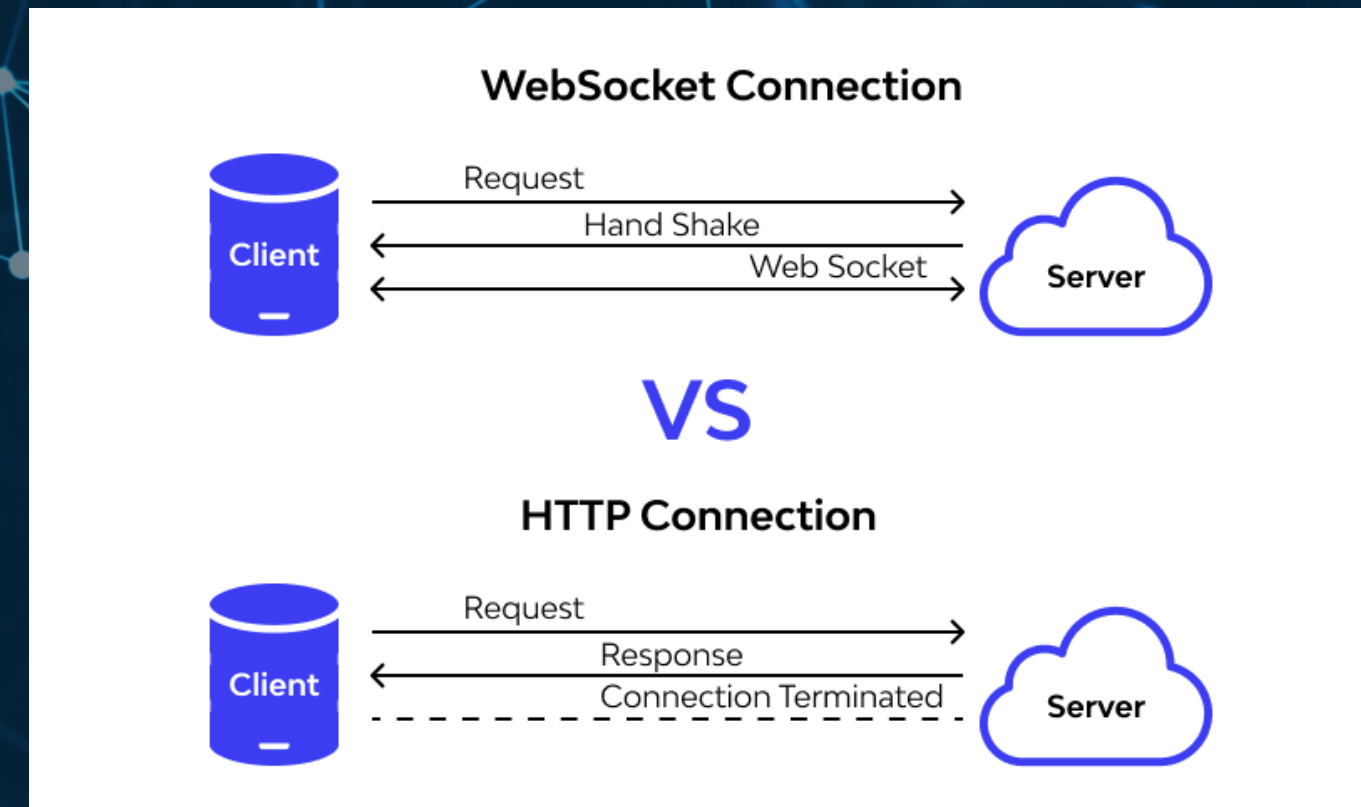


Server Sent Events (SSE)



SSE (Server-Sent Events):

Utilizamos **Server-Sent Events** para mantener informados a los ciudadanos sobre el estado de sus reportes. Cuando un supervisor cambia una incidencia de **Pendiente a En progreso o Resuelto**, el servidor envía automáticamente la actualización al cliente, sin necesidad de que la aplicación consulte constantemente si hubo cambios.



Websockets:

Utilizamos **WebSocket** para la optimización de rutas en tiempo real. La aplicación del supervisor envía continuamente su ubicación GPS al servidor mientras se desplaza. Con esa información, el servidor puede recalcular la ruta si aparecen nuevas incidencias o cambia el recorrido, devolviendo inmediatamente la información actualizada. Esta es una comunicación bidireccional y de baja latencia.



03

Desarrollo Frontend

FICHA TÉCNICA DEL FRONTEND



CARACTERÍSTICAS DESTACADAS Y DIFERENCIADORES TÉCNICOS

- Mapas inteligentes y dinámicos



• Sistema de navegación y seguimiento “UBER-STYLE”



- **Reactividad en Tiempo Real sin Sobrecargar**



ESTRUCTURA DEL CÓDIGO DEL PROYECTO





04

Patrones de Diseño

Patrón Strategy

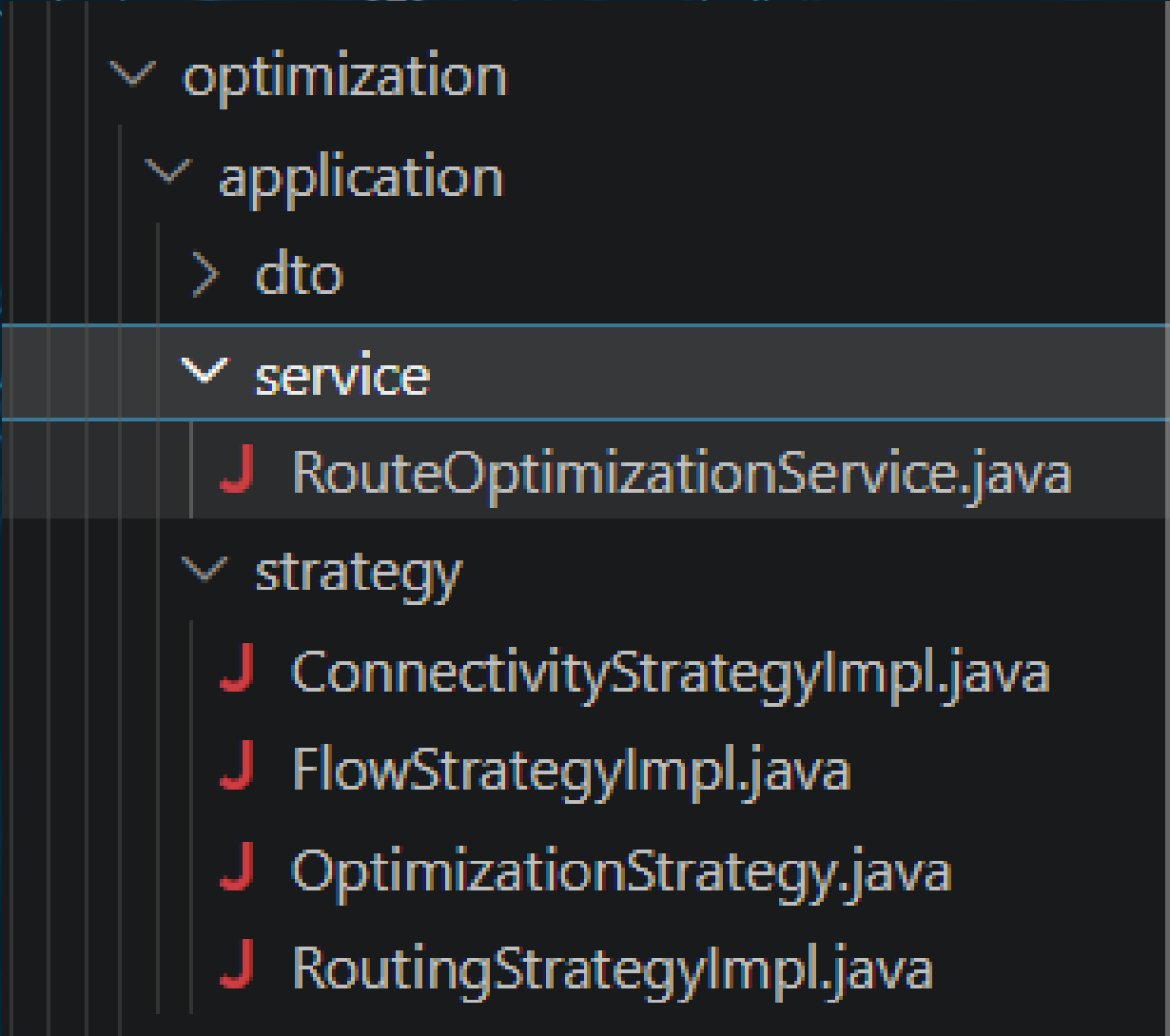
En primer lugar, utilizamos el patrón Strategy en el módulo de optimización. Definimos una interfaz común llamada **OptimizationStrategy** y contamos con tres implementaciones.

RoutingStrategy utiliza el algoritmo de vecino más cercano ponderado para ordenar la atención de incidencias.

ConnectivityStrategy emplea el algoritmo de Prim para conectar puntos con la menor distancia total.

FlowStrategy utiliza Ford-Fulkerson con Edmonds-Karp para representar redes de flujo.

El servicio principal recibe el tipo de algoritmo asociado a la categoría y selecciona automáticamente la estrategia correspondiente.



```

└─ optimization
   └─ application
      └─ dto
         └─ service
            └─ RouteOptimizationService.java
               └─ strategy
                  └─ ConnectivityStrategyImpl.java
                     └─ FlowStrategyImpl.java
                        └─ OptimizationStrategy.java
                           └─ RoutingStrategyImpl.java
```


Patrón State

El propósito del patrón state es permitir que un objeto cambie su comportamiento cuando su estado interno cambia.

Problema: Los reportes pasan por fases (Pendiente, En Progreso, Resuelto). Cada fase restringe ciertas acciones (por ejemplo, no se puede editar un reporte Resuelto). Utilizar banderas booleanas u operadores espagueti.

Solución: Se modelaron clases discretas de estados (PendingState, ResolvedState). Ahora, el objeto delega la validación directamente a su estado actual.

```
ReportStatusFactory.java X
backend > src > main > java > com > reportatucalle > modules > report > domain > e
1 package com.reportatucalle.modules.report.domain.entity;
2
3 public class ReportStatusFactory {
4     public static ReportStatus fromString(String status) {
5         if (status == null) return new PendingState();
6         return switch (status.toUpperCase()) {
7             case "PENDING" -> new PendingState();
8             case "ASSIGNED" -> new AssignedState();
9             case "IN_PROGRESS" -> new InProgressState();
10            case "RESOLVED" -> new ResolvedState();
11            case "REJECTED" -> new RejectedState();
12            default -> new PendingState();
13        };
14    }
15 }
16
```

```
package com.reportatucalle.modules.report.domain.entity;

public class ResolvedState implements ReportStatus {
    @Override
    public String getName() {
        return "RESOLVED";
    }

    @Override
    public boolean canUpdate() {
        return false;
    }

    @Override
    public boolean isActive() {
        return false;
    }
}
```

Patrón Observer

Propósito: Definir una dependencia uno-a-muchos entre objetos de manera que, cuando uno cambie de estado, todos sus dependientes sean notificados.

Problema: Tras registrarse un incidente, múltiples submódulos (Auditoría, Analíticas de Calor, Notificaciones) necesitaban reaccionar independientemente sin que el módulo creador los conociera explícitamente.

Solución y Explicación: Un mecanismo de eventos. El EventPublisher gestiona una lista de oyentes genéricos.

```
1 package com.reportatucalle.shared.event;
2
3 import org.springframework.stereotype.Component;
4
5 import java.util.ArrayList;
6 import java.util.List;
7
8 public class EventPublisher<T> {
9     private final List<EventListener<T>> listeners = new ArrayList<>();
10
11     public void subscribe(EventListener<T> listener) {
12         listeners.add(listener);
13     }
14
15     public void unsubscribe(EventListener<T> listener) {
16         listeners.remove(listener);
17     }
18
19     public void publish(T event) {
20         for (EventListener<T> listener : listeners) {
21             listener.onEvent(event);
22         }
23     }
24 }
25
```


Llamada a eventPublisher

```
backend > src > main > java > com > reportatucalle > modules > report > application > service > ReportService.java > ReportService
39 public class ReportService implements IReportService {
64     public ReportResponse createReport(CreateReportRequest request) {
124
125         return reportMapper.toResponse(existingReport);
126     }
127
128     // No existe duplicado, crear nuevo reporte
129     Report newReport = reportMapper.toEntity(request, citizenProfile.getId());
130     Report savedReport = reportRepository.save(newReport);
131
132     notificationPort.notifyReportCreated(savedReport);
133     eventPublisher.publish(new ReportCreatedEvent(savedReport));
```



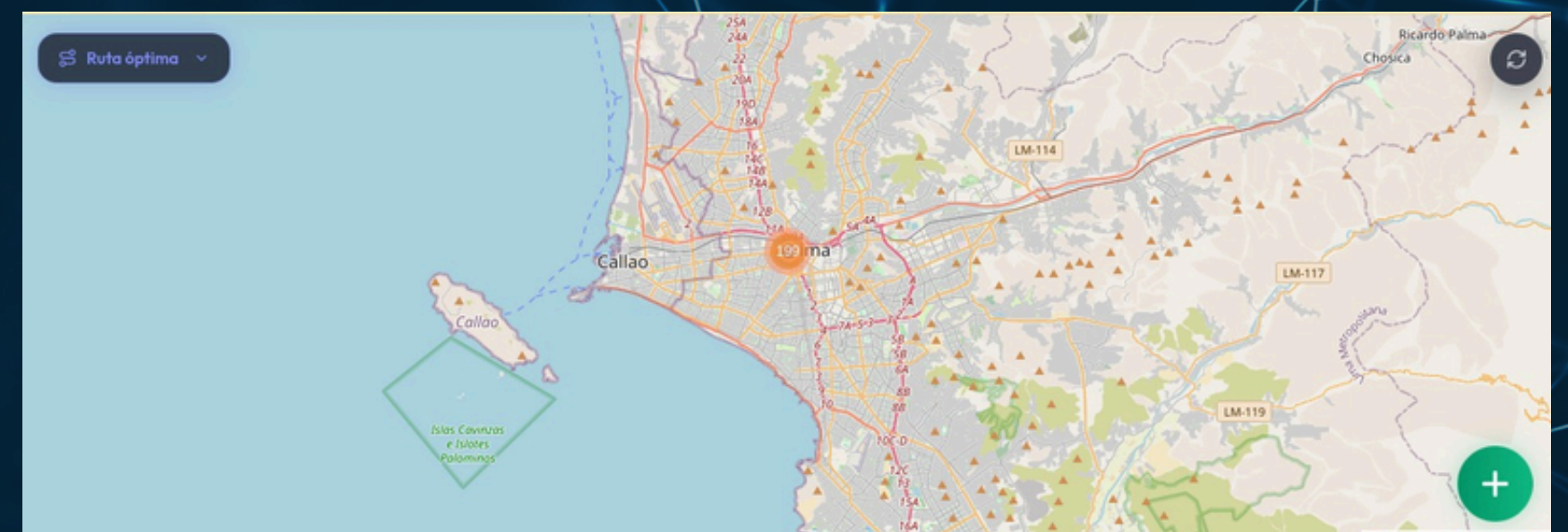
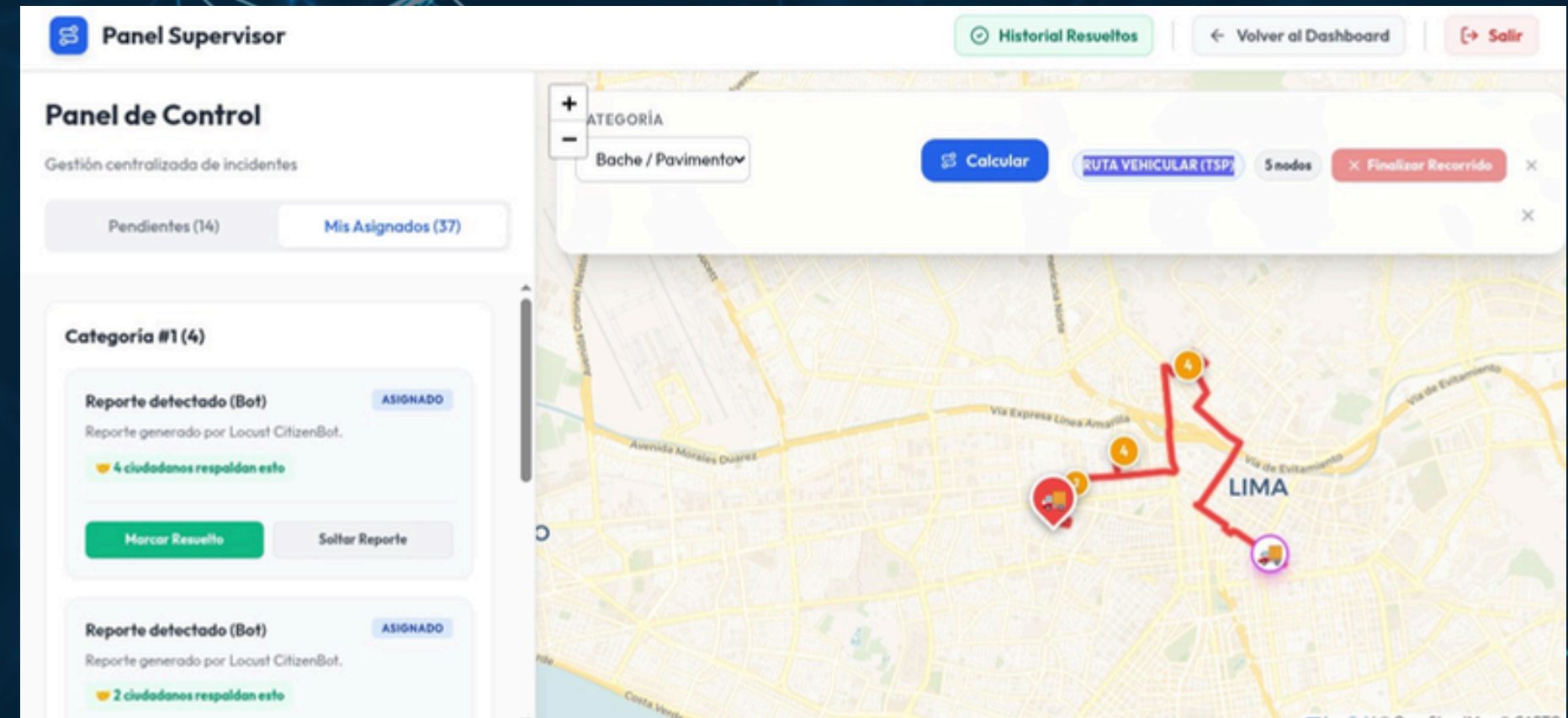

05

Estrategia de Testing

Locust

Locust es una herramienta de pruebas de carga. Sirve para simular una gran cantidad de usuarios utilizando la aplicación al mismo tiempo. Esto se puede usar para probar rutas, clústeres, etc.

```
C: > Users > Arturo > Desktop > ReportaTuCalle-main (1) > ReportaTuCalle-main > locustfile.py > ...
1 from locust import HttpUser, task, between
2 import random
3 import string
4
5 class CitizenBot(HttpUser):
6     weight = 10
7     wait_time = between(1, 3)
8     host = "http://localhost:8080"
9
10    def on_start(self):
11        random_string = ''.join(random.choices(string.ascii_lowercase, k=8))
12        self.email = f"citizen_{random_string}@test.com"
13        self.password = "Password123!"
14
15
16        self.client.post("/api/v1/auth/register", json={
17            "email": self.email,
18            "password": self.password,
19            "firstName": "CitizenBot",
20            "lastName": random_string
21        })
```



Cypress

Cypress simula el comportamiento de un usuario real (endpoint a endpoint) para verificar las funcionalidades principales de la aplicación.

The image displays the Cypress Studio interface on the left and the application under test on the right.

Cypress Studio (Left):

- File Explorer:** Shows `auth.cy.js`.
- Test Runner:** Shows 1 passed and 1 failed test.
- Test Suite:** "Autenticación y Sesión".
- Test Case:** "Debe mostrar error con credenciales inválidas".
- Test Body:**

```
1 visit /login
2 get input[type="email"]
3   .type('test@fake.com')
4 get input[type="password"]
5   .type('wrongpass')
6 get button[type="submit"]
7   .click()

(xhr) POST 401 http://localhost:8080/api/v1/auth/login loginError
8 wait @loginError
9   .contains('Correo o contraseña incorrectos')
10  .assert('expected <p.flex-1.loading-snug> to be visible')
```

Application (Right):

- URL:** `http://localhost:5173/login`
- Page Title:** "Iniciar sesión"
- Form Fields:** "Correo electrónico" (containing `test@fake.com`) and "Contraseña" (containing `wrongpass`).
- Buttons:** "Iniciar sesión" and "¿No tienes cuenta? Regístrate aquí".
- Error Message:** "Correo o contraseña incorrectos. Intenta de nuevo."
- Footer:** "© 2024 Municipalidad - ReportaTuCalle"



06

Demostración del aplicativo

Supabase



Se utilizó para almacenar toda la información del sistema, como usuarios, reportes, categorías y estados. El backend de Spring Boot se conecta a esta base de datos mediante JPA/Hibernate para guardar y consultar los datos.

Vercel

Se utilizó para publicar la interfaz web de ReportaTuCalle, permite que cualquier usuario acceda a la aplicación desde un navegador mediante una URL pública.

Cada vez que se actualiza el código en GitHub, Vercel puede compilarlo y desplegar automáticamente la nueva versión.



Link de GitHub



<https://github.com/Inka-Innovations-S-A-C/ReportaTuCalle>

Link de la App

reportatucalle-vercel.vercel.app

Inka-Innovations-S-A-C/ **ReportaTuCalle**



Repositorio de la aplicación ReportaTuCalle, se seguirá la metodología ágil de Scrum durante el desarrollo del proyecto.

 5
Contributors

 0
Issues

 0
Stars

 0
Forks



Inka-Innovations-S-A-C/ReportaTuCalle: Repositorio de la aplicación ReportaTuCalle, se seguirá la metodología ágil de Scrum durante el...

Repositorio de la aplicación ReportaTuCalle, se seguirá la metodología ágil de Scrum durante el desarrollo del proyecto. - Inka-Innovations-S-A-C/ReportaTuCalle

 GitHub



¡Gracias por la atención!